

# Backend Fundamentals — Recap 5

Idempotency keys, and why naive pagination breaks

Status: idempotency keys fully implemented and tested. Pagination is mid-way — the concept and the bug are understood; the cursor-based code itself still needs to be rebuilt slowly from here.

## 1. Idempotency Keys — Making POST Safe to Retry

GET, PUT, and DELETE are idempotent — repeating them with the same input leaves things in the same state. POST is not — repeating it creates a duplicate. A real-world failure case: a client sends POST, the server creates the resource, but the response is lost (e.g. a network blip) before the client sees it. The client retries, not knowing the first attempt actually succeeded — and now two resources exist instead of one.

Fix: the client generates a unique key before sending, attached as a header (Idempotency-Key: abc-123). The server remembers which keys it has already handled and what it responded with. A repeated key returns the exact same saved response instead of creating anything new.

```
const idempotencyStore = new Map();

app.post('/tasks', (req, res) => {
  const idempotencyKey = req.headers['idempotency-key'];

  if (idempotencyKey && idempotencyStore.has(idempotencyKey)) {
    const cached = idempotencyStore.get(idempotencyKey);
    return res.status(cached.status).send(cached.body);
  }

  // ...validation + insert as before...

  if (idempotencyKey) {
    idempotencyStore.set(idempotencyKey, { status: 201, body: newTask });
  }
  res.status(201).send(newTask);
});
```

Confirmed by testing: same key sent twice → only one task created, second response was the cached one. No key at all → every request treated as new, duplicates appear freely, same as before idempotency was added.

Also confirmed: the key is checked before the body is ever read, so a repeated key returns the original response even if the title in the second request is different. This is intentional — the key represents one specific attempt; the API's job is to guarantee no duplicate action, not to guess which version of the data was "really" meant.

## 2. A Third Source of Request Data: req.query

Three separate places data can come from in a request, now all covered:

```
req.params    // pieces baked into the URL path itself, e.g. the 1 in /tasks/1
req.body      // data sent in a POST/PUT body, e.g. { "title": "..."}
req.query     // the ?key=value part of a URL, e.g. ?after=10 in /tasks?after=10
```

`req.query` exists because GET requests have no body to carry extra data in — the query string is how a GET request still passes along extra information, like which page or cursor to fetch.

### 3. The Ternary Operator — Compact if/else

```
const after = req.query.after ? Number(req.query.after) : 0;
```

```
// equivalent to:
let after;
if (req.query.after) {
  after = Number(req.query.after);
} else {
  after = 0;
}
```

Read as: "if a value was given, use it (converted to a number); otherwise default to 0."

### 4. Why "Just Send Everything" Doesn't Scale

Returning every row from GET `/tasks` is fine with a handful of tasks, but becomes a huge, slow response once a table has thousands or millions of rows. Pagination means sending back a small chunk at a time instead of the whole table.

### 5. Offset Pagination, and Where It Breaks

The naive approach: "page 2" means skip the first  $N$  rows, then take the next  $N$  (`LIMIT ? OFFSET ?` in SQL). The problem is that this counts *positions*, not specific items — and positions shift whenever something is added or deleted.

**The raffle-line analogy:** ten people stand in a numbered line. Tickets are handed to people 1-5 ("page 1"). Before the next batch, person #3 leaves — everyone behind shifts forward by one spot. Handing out the "next 5" now means someone who already got a ticket (shifted into a new-looking spot) gets counted again, while someone further back never gets counted at all. Nothing alerts anyone that this happened — it fails silently.

Demonstrated directly: fetching offset-based page 1 and page 2, deleting an item that had appeared on page 1, then re-fetching page 2 was set up to show the shift in ids — confirming the same failure the raffle-line analogy describes.

### 6. Cursor Pagination — The Fix (Concept)

Instead of "skip  $N$  positions," the client says "give me what comes after this specific item I already saw" — identified by its actual id, not by a count. Since an id is a fixed identity rather than a shifting position, deleting or adding items elsewhere doesn't throw off what "after  $X$ " means.

```
// shape of the idea – to be rebuilt step by step next session  
SELECT * FROM tasks WHERE id > ? ORDER BY id LIMIT ?
```

The SQL query itself was tested earlier and worked correctly (fetching pages via `?after=`), but the full explanation hadn't landed yet — next step is rebuilding this slowly, piece by piece, rather than reusing the earlier code as-is.

---

Next up: rebuild cursor-based pagination from scratch, one small piece at a time, then API versioning.